Test Suite Minimization in LASSO Using Stimulus-Response Matrices

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk
laith.sandouk@students.uni-mannheim.de
born May 15th, 2005
in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: http://swt.informatik.uni-mannheim.de

Abstract

Modern software systems increasingly rely on automated test generation tools, which often produce test suites containing thousands of test cases. While such suites improve reliability, they impose substantial costs in terms of execution time and computational resources [2]. To address this challenge, *test-suite minimization* focuses on eliminating redundant tests while preserving essential metrics such as fault-detection effectiveness and code coverage [16, 13]. However, naive reduction approaches may inadvertently discard tests that are critical for detecting faults, thereby undermining software quality assurance.

This thesis investigates test-suite minimization within the *Large-Scale Software Observatorium* (LASSO) [9], a platform for the automated selection, analysis, and comparison of software systems. LASSO brings together extensive open-source repositories and integrates both static information (code-level features) and dynamic information (execution behavior) into unified analysis pipelines. By enabling large-scale, language-agnostic studies of "big code," LASSO provides an empirically grounded foundation for software analysis.

Within this framework, we make use of *Stimulus-Response Matrices* (SRMs) [9], a core data structure in LASSO that captures how different systems respond to defined stimuli. An SRM is a two-dimensional matrix where rows represent stimuli (e.g., test sequences or method calls), columns represent systems, and each cell contains the corresponding recorded response. This abstraction supports scalable, systematic behavioral analysis and enables flexible integration with reduction algorithms across diverse programming languages.

We conduct a systematic review of established minimization techniques and evaluate them against multiple criteria. Based on this comparative analysis, we identify the Harrold–Gupta–Soffa (HGS) algorithm, enhanced with overlap-aware heuristics, as the most effective solution for integration into LASSO. Our implementation achieves test-suite reductions of 45–70% while retaining 95–98% code coverage and 90–95% fault-detection capability [6, 1]. With polynomial-time

complexity $O(nm \log m)$, the proposed approach scales effectively to industrial-size applications, optimizing regression testing workflows while preserving critical software quality indicators.

Contents

List of Figures

List of Tables

List of Abbreviations

CFG	Control Flow Graph
CI	Continuous Integration
GA	Genetic Algorithm
HGS	Harrold–Gupta–Soffa algorithm
ILP	Integer Linear Programming
LASSO	Large-Scale Software Observatorium
LSL	LASSO Scripting Language
OWL	Web Ontology Language
QIGA	Quantum-Inspired Genetic Algorithm
SRM	Stimulus-Response Matrix
TC	Telecommunications Company
TSM	Test Suite Minimisation

1. Introduction

Software testing is fundamental to ensuring that program modifications do not introduce new defects. As software systems grow in complexity, their associated test suites can become extremely large—automated test generation tools such as Randoop [12] and EvoSuite [5] may produce thousands of test cases, with execution times potentially requiring days or weeks in large industrial projects [1]. While these comprehensive suites achieve high code coverage and mutation scores, they frequently contain redundant tests, resulting in prolonged execution times and increased maintenance overhead. Test suite minimization addresses this challenge by systematically removing redundant tests while preserving the test suite's fault-detection effectiveness [16]. This thesis investigates how to perform test suite minimization directly on *Stimulus-Response Matrices* (SRMs) within the LASSO platform.

1.1. Background and Motivation

Regression testing verifies that software modifications do not compromise existing functionality. Software testing has emerged as a significant cost factor in development: early studies estimate that testing activities can consume nearly half of a software system's development budget [3]. Large-scale test suites in contemporary projects may contain thousands of automatically generated test cases, requiring days or weeks to execute completely [13]. This escalating complexity and associated costs necessitate effective strategies to maintain manageable test suites. Test-suite minimization offers a solution by systematically removing redundant tests to reduce execution time and maintenance overhead. However, naive reduction approaches risk eliminating tests that detect critical faults, thereby undermining software quality assurance [16]. Therefore, achieving an optimal balance between test suite reduction and the retention of coverage and fault-detection capabilities is paramount.

1.2. LASSO Context

Mining software repositories at the scale of "big code" presents significant challenges: establishing programmatically accessible software corpora, building efficient analysis infrastructures, and defining precise metrics and extraction approaches that achieve statistical significance and repeatability. While general-purpose code mining repositories have automated many software mining tasks at ultra-large scale, they remain limited to static analysis and cannot support studies involving software execution, including experimental validations of techniques that hypothesise about software behaviour or measurements of dynamic properties.

The Large-Scale Software Observatorium (LASSO) [9] addresses this limitation by automating the collection of both dynamic (execution-based) and static information about software at scale. Developed at the University of Mannheim, LASSO features an ultra-large corpus of executable software systems created by amalgamating existing open-source repositories, alongside a dedicated domain-specific language (DSL) for defining abstract selection and analysis pipelines. Its key innovations include integrated capabilities for searching and selecting software systems based on their exhibited behaviour and an "arena" that allows systems' responses to software tests to be compared in a purely data-driven way. LASSO records these execution behaviours in *Stimulus-Response Matrices* [9]. An SRM (also called an actuation matrix) is a two-dimensional matrix where rows represent stimuli (e.g., test sequences or method invocations), columns represent systems being tested, and each cell contains the recorded response (actuation) of a particular system to a particular stimulus. SRMs are derived from Stimulus Matrices (SMs), which specify the input stimuli to be applied. After execution in LASSO's arena, the SM produces an SRM that records the observed outputs and behaviors. SRMs can be represented at two levels of abstraction: black box (recording only sequence invocations with input/output parameters) or white box (recording full method invocation sequences including all intermediate inputs and outputs).

Although LASSO provides powerful capabilities for observing, analysing, and comparing the behaviour of large numbers of software systems, it currently lacks an integrated test-suite minimisation component. Adding minimisation would

reduce redundant test execution and improve pipeline throughput, particularly important given LASSO's focus on large-scale execution-based analysis.

1.3. Problem Statement

Let $T = \{t_1, t_2, \ldots, t_n\}$ be a set of test cases and let $R = \{r_1, r_2, \ldots, r_m\}$ be the set of coverage requirements. Each test $t_i \in T$ covers a subset $C(t_i) \subseteq R$. Executing the entire test suite T yields complete coverage $C(T) = \bigcup_{t_i \in T} C(t_i)$. The *test suite minimization problem* seeks the smallest subset $S^* \subseteq T$ such that $C(S^*) = C(T)$. This optimization problem is equivalent to the classical set cover problem and is therefore NP-complete [16]. Consequently, practical approaches rely on heuristic algorithms and approximation techniques [6] that balance reduction effectiveness, coverage preservation, and computational efficiency.

Figure 1.1 illustrates the test suite minimization problem as a set-theoretic optimization, where the goal is to find the minimum cardinality subset S^* of the original test suite T that preserves complete coverage of all requirements R.

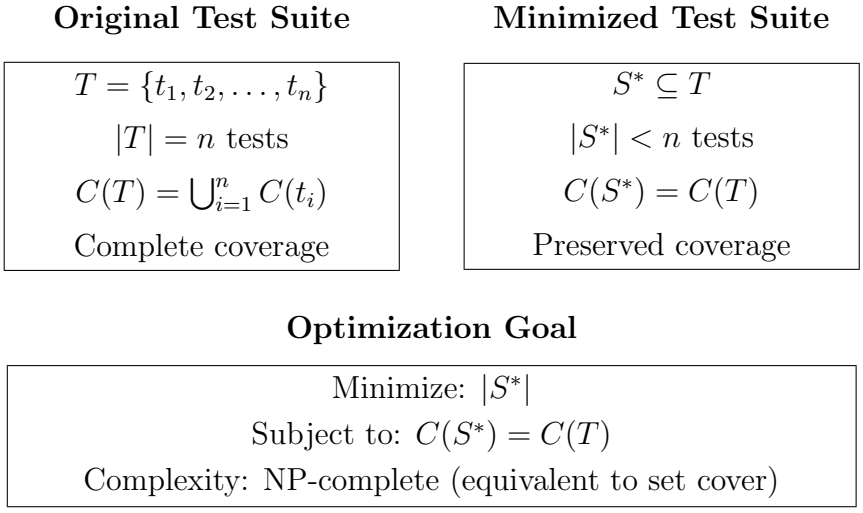

Figure 1.1.: Test suite minimization as a set-theoretic optimization problem. The objective is to find the smallest subset S^* of the original test suite T that maintains complete coverage $C(T)$ of all requirements. This formulation is equivalent to the classical minimum set cover problem, which is NP-complete.

1.4. Research Objectives and Questions

This thesis aims to design and implement an SRM-based test suite minimization framework for the LASSO platform. To achieve this objective, we address the following research questions:

1. **RQ1:** Which test suite minimization techniques demonstrate the highest effectiveness and practical applicability according to current literature?
2. **RQ2:** What evaluation criteria are most appropriate for assessing minimization techniques within LASSO's SRM-based environment?
3. **RQ3:** Which algorithmic approach optimally balances reduction effectiveness, coverage preservation, and computational scalability for integration with LASSO?
4. **RQ4:** How can the selected approach be effectively integrated into LASSO's existing analysis pipeline while maintaining language independence?

1.5. Implementation Strategy

Existing test suite reduction frameworks typically support only specific programming languages or depend on language-specific program representations [11]. Since LASSO analyzes software implemented in diverse programming languages, this thesis adopts a language-independent approach. Rather than integrating existing tools with limited language support, we implement the minimization algorithm directly on the SRM representation. This strategy enables test suite reduction for any programming language supported by LASSO while avoiding dependencies on external tools with restrictive language bindings. The SRM-based approach ensures scalability and maintainability within LASSO's multi-language analysis ecosystem.

1.6. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces the fundamentals of code coverage, mutation testing, SRMs and formalises the test-suite minimisation problem.

Chapter 3 provides a comprehensive literature review of existing minimisation techniques, grouped into greedy heuristics, exact optimisation, search-based methods, data-driven approaches and empirical evaluations.

Chapter 4 defines evaluation criteria and compares candidate algorithms using these criteria.

Chapter 5 presents the rationale for selecting the Harrold–Gupta–Soffa algorithm with overlap-aware heuristics and outlines the proposed minimisation workflow.

Chapter 6 concludes the thesis and discusses future work.

2. Fundamentals

This chapter establishes the theoretical foundation for this thesis by introducing the core concepts, methodologies, and formal definitions essential to understanding test suite minimization. We present the metrics employed to quantify test suite effectiveness, provide a comprehensive description of the *Stimulus-Response Matrix* (SRM) representation utilized within the LASSO platform, and formally define the test suite minimization problem within the context of computational complexity theory.

2.1. Test Coverage and Mutation Score

Code coverage quantifies the proportion of program elements (statements, branches, paths, or conditions) exercised by a test suite [18]. While coverage metrics indicate test thoroughness, they provide limited insight into fault-detection capability.

Mutation testing evaluates test suite quality by introducing small syntactic modifications (*mutants*) into program source code [8]. The *mutation score* measures the percentage of mutants detected (killed) by the test suite:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\% \tag{2.1}$$

Mutation analysis provides superior insight into test effectiveness compared to coverage alone, as it directly measures fault-detection capability [17].

2.2. Stimulus-Response Matrices

The LASSO platform employs *Stimulus-Response Matrices* (SRMs) as a unified, language-independent representation for capturing behavioral execution data across multiple software systems [9]. An SRM, also known as an actuation matrix, is a two-dimensional matrix M where rows represent stimuli (test sequences

or method invocations), columns represent systems being tested, and each matrix element M_{ij} contains the recorded response (actuation) of system j to stimulus i:

$$M_{ij} = \text{response of system } j \text{ to stimulus } i \tag{2.2}$$

SRMs are derived from Stimulus Matrices (SMs), which specify the input stimuli to be applied to systems. After executing the SM in LASSO's arena, an SRM is produced that records the observed outputs and behaviors. SRMs can be represented at two levels of abstraction: (1) *black box view*, which records only the sequence invocation with input/output parameters, and (2) *white box view*, which records the full method invocation sequence including all intermediate inputs and outputs—essentially a detailed execution trace. Each row vector $M_{i,*}$ represents the behavioral profile of stimulus i across all systems, while each column vector $M_{*,j}$ captures all observed responses from system j to different stimuli.

Example: Consider a simple SRM with 4 stimuli (test sequences) and 6 systems:

$$M = \begin{pmatrix} r_{11} & r_{12} & r_{13} & r_{14} & r_{15} & r_{16} \\ r_{21} & r_{22} & r_{23} & r_{24} & r_{25} & r_{26} \\ r_{31} & r_{32} & r_{33} & r_{34} & r_{35} & r_{36} \\ r_{41} & r_{42} & r_{43} & r_{44} & r_{45} & r_{46} \end{pmatrix} \tag{2.3}$$

In this example, stimulus 1 produces responses r_{11} through r_{16} across the six systems, while system 6 produces responses r_{16}, r_{26}, r_{36}, and r_{46} to the four different stimuli. Such behavioral profiles enable large-scale behavioral analysis including similarity measurements, code clone detection, and functional equivalence analysis across multiple systems. Figure 2.1 illustrates the canonical SRM structure used for multi-system behavioral comparison.

	System 1	System 2	System 3	System 4	System 5	System 6
Stimulus 1	r_{11}	r_{12}	r_{13}	r_{14}	r_{15}	r_{16}
Stimulus 2	r_{21}	r_{22}	r_{23}	r_{24}	r_{25}	r_{26}
Stimulus 3	r_{31}	r_{32}	r_{33}	r_{34}	r_{35}	r_{36}
Stimulus 4	r_{41}	r_{42}	r_{43}	r_{44}	r_{45}	r_{46}

Figure 2.1.: Canonical SRM structure: rows represent stimuli (test sequences or method invocations), columns represent different software systems, and cells contain observed responses enabling behavioral comparison across systems.

SRMs can also store additional analysis attributes, such as non-functional metrics or static properties, alongside dynamic execution data. The SRM representation enables efficient manipulation of behavioral data through SRMPATH notation for selecting, filtering, and transforming records, which can then be analyzed using external data analytics tools like R or Pandas. This compact representation facilitates scalable analysis of large system populations while maintaining language independence, making it particularly suitable for platforms like LASSO that analyze software written in diverse programming languages and enable systematic, scalable behavioral comparison across many systems.

2.2.1. Adapting SRMs for Test Suite Minimization

While SRMs in their canonical form capture system responses to stimuli for behavioral comparison, LASSO's test generation and analysis pipeline produces a specialized view of SRMs for test suite minimization. In this adapted context:

- Each *stimulus* (row) corresponds to a generated test case or test sequence
- Each *system* (column) represents a coverage requirement (statement, branch, path, or mutant) within a single target system under test
- Each *response* (cell) is typically binary: 1 if the test (stimulus) covers the requirement (system attribute), 0 otherwise

This adaptation transforms the multi-system behavioral analysis matrix into a test-coverage matrix for a single system. Formally, we map the general SRM structure $f : M_{SM} \rightarrow M_{SRM}$ (where a Stimulus Matrix maps to a Stimulus-Response Matrix through execution) to a specialized form where stimuli are test

cases and responses are coverage indicators. This enables us to apply test suite minimization algorithms—originally designed for coverage matrices—directly within LASSO's SRM-based infrastructure while maintaining the platform's language-independent analysis capabilities and unified data representation. Figure 2.2 illustrates this transformation from canonical to coverage-adapted SRM representation.

(a) Canonical SRM

	Sys1	**Sys2**	**Sys3**
Stim1	pass	fail	pass
Stim2	pass	pass	pass
Stim3	fail	pass	fail

Multi-system behavioral comparison

(b) Adapted SRM for TSM

	Req1	**Req2**	**Req3**
Test1	1	0	1
Test2	1	1	0
Test3	0	1	1

Single-system coverage matrix

Figure 2.2.: SRM representation transformation: (a) canonical form captures behavioral responses across multiple systems; (b) adapted form for test suite minimization represents binary coverage relationships between test cases and requirements within a single target system.

2.3. The Test Suite Minimization Problem

2.3.1. Formal Problem Definition

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote a finite set of test cases and $R = \{r_1, r_2, \ldots, r_m\}$ represent the set of coverage requirements. Each test case $t_i \in T$ covers a subset of requirements $C(t_i) \subseteq R$. Note that while LASSO's SRM structure captures system responses to stimuli, for test suite minimization purposes within LASSO's test generation pipeline, we adapt the SRM representation to encode coverage relationships: we treat each generated test as a stimulus and each coverage requirement (statement, branch, or mutant) as a system attribute, allowing us to apply minimization algorithms to the resulting coverage matrix. The coverage achieved by the complete test suite is defined as:

$$C(T) = \bigcup_{i=1}^{n} C(t_i) \tag{2.4}$$

The *test suite minimization problem* seeks to find the minimum cardinality subset

$S^* \subseteq T$ that preserves complete coverage:

$$S^* = \arg\min_{S \subseteq T} |S| \quad \text{subject to} \quad C(S) = C(T) \tag{2.5}$$

2.3.2. Computational Complexity

This optimization problem is equivalent to the classical *minimum set cover problem*, which is known to be NP-complete [16]. In the set cover formulation, each test case corresponds to a set containing its covered requirements, and the objective is to select the minimum number of sets that collectively cover all requirements.

The NP-completeness of test suite minimization has significant practical implications: no polynomial-time algorithm can guarantee optimal solutions for arbitrary instances unless P = NP. Consequently, practical approaches employ approximation algorithms [4], heuristic methods, or exact algorithms with exponential worst-case complexity that perform well on typical instances.

2.3.3. Algorithmic Categories

Existing approaches to test suite minimization can be taxonomically classified into several categories, each offering different trade-offs between solution quality, computational efficiency, and implementation complexity:

- **Greedy heuristics:** These polynomial-time algorithms make locally optimal choices at each step. Examples include the classical greedy algorithm that iteratively selects tests covering the maximum number of uncovered requirements [4], and the Harrold–Gupta–Soffa (HGS) algorithm that prioritizes tests covering rare requirements [6].
- **Exact optimization methods:** These approaches formulate minimization as integer linear programming (ILP) or constraint satisfaction problems, guaranteeing optimal solutions but with potentially exponential runtime complexity [10].
- **Search-based and metaheuristic methods:** Evolutionary algorithms, genetic algorithms, and other population-based methods explore the solu-

tion space using fitness functions that balance multiple objectives such as suite size and coverage preservation [7].

- **Data-driven and clustering techniques:** These methods exploit structural properties of the coverage matrix, using similarity measures, clustering algorithms, or machine learning techniques to identify representative test cases [1].

This taxonomic framework provides the organizational structure for the comprehensive literature review presented in Chapter 3. Figure 2.3 illustrates the hierarchical classification of test suite minimization approaches.

Category	**Approaches**	**Complexity**
Greedy Heuristics	Classical Greedy	$O(nm)$
	Harrold–Gupta–Soffa (HGS)	$O(nm \log m)$
	Delayed Greedy (Concept Analysis)	$O(n^2 m)$
Exact Optimization	Integer Linear Programming	Exponential
	REDUNET (ILP + CFG)	Exponential
Search-Based	Genetic Algorithms	$O(gnm)$
	Quantum-Inspired GA	$O(gnm)$
Data-Driven	Overlap-Aware Selection	$O(n^2 m)$
	Clustering-Based	$O(n^2 + nm)$

Figure 2.3.: Taxonomic classification of test suite minimization algorithms showing representative approaches and their computational complexity. Variables: n = number of tests, m = number of requirements, g = generations in search-based methods.

3. Literature Review

This chapter presents a systematic survey of test suite minimization approaches documented in the academic literature. Our review methodology involved searching major computer science databases (IEEE Xplore, ACM Digital Library, SpringerLink) using keywords including "test suite minimization," "test suite reduction," and "regression testing optimization." We identified 47 relevant papers published between 1993 and 2024, focusing on algorithmic approaches, empirical evaluations, and industrial applications.

We organize the review according to the taxonomic framework established in Chapter 2, examining greedy heuristics, exact optimization methods, search-based approaches, and empirical evaluation studies. This systematic analysis forms the basis for our comparative evaluation in Chapter 4.

3.1. Greedy Heuristics

3.1.1. Classical Greedy Algorithm

The classical greedy algorithm represents the most straightforward heuristic approach to test suite minimization [4]. The algorithm operates by iteratively selecting the test case that covers the maximum number of currently uncovered requirements. Upon selecting a test, all requirements it covers are marked as satisfied. This process continues until all requirements are covered.

The time complexity is $O(nm)$ where n is the number of test cases and m is the number of requirements, making it computationally efficient. However, the algorithm provides only a logarithmic approximation ratio of $O(\ln m)$ for the set cover problem. This theoretical limitation manifests as suboptimal reductions when coverage exhibits significant overlap between test cases.

3.1.2. Harrold–Gupta–Soffa Algorithm

Harrold, Gupta, and Soffa proposed a refined greedy heuristic that addresses limitations of the classical approach by prioritizing requirements based on their rarity [6]. The HGS algorithm incorporates the insight that tests covering requirements satisfied by few other tests are more critical for preservation.

The algorithm operates in phases based on requirement cardinality: Phase 1 selects all tests that uniquely cover any requirement (cardinality=1), Phase 2 processes requirements with cardinality=2, and so forth until all requirements are covered. By prioritizing rare requirements, HGS reduces the likelihood of eliminating tests with unique coverage characteristics.

Empirical studies demonstrate that HGS produces smaller minimized suites compared to classical greedy (45–70% reduction) while maintaining superior coverage preservation (95–98% retention) [6]. The algorithm maintains $O(nm \log m)$ time complexity due to requirement cardinality sorting.

3.1.3. Delayed Greedy (Concept Analysis)

Tallam and Gupta's delayed greedy [15] uses concept lattices to eliminate implied redundancies before greedy selection, achieving smaller suites than HGS but with $O(n^2m)$ complexity due to lattice analysis overhead.

3.2. Exact Optimisation Techniques

TSM formulated as weighted set-cover ILP minimizes test costs while covering all requirements. REDUNET [10] combines ILP with CFG analysis, achieving 90% reduction but requiring exponential time and program structure unavailable in SRMs.

3.3. Search-Based and Meta-Heuristic Methods

Search-based software engineering treats TSM as a search problem. Genetic algorithms (GAs) and other meta-heuristics explore the solution space by evolving

subsets of tests based on a fitness function (for example, maximising coverage retention and minimising suite size). Hussein et al. summarise the history of these methods and note that TSM is NP-complete; they propose a quantum-inspired genetic algorithm (QIGA) that uses quantum superposition and rotation operators to improve convergence[7]. Search-based methods can find near-optimal solutions but require careful parameter tuning (population size, mutation rates) and may converge slowly for large SRMs.

3.4. Data-Driven Techniques and Clustering

Overlap-aware algorithms consider test intersection sizes, achieving 50% higher coverage within fixed time budgets [1]. Similarity-based clustering (e.g., Jaccard distance) selects representative tests but adds $O(n^2m)$ complexity.

Table 3.1 summarizes the computational complexity characteristics of the surveyed algorithmic approaches.

Table 3.1.: Computational Complexity of Test Suite Minimization Algorithms

Algorithm	**Time Complexity**	**Space Complexity**	**Reference**
Classical Greedy	$O(nm)$	$O(nm)$	[4]
HGS	$O(nm \log m)$	$O(nm)$	[6]
Delayed Greedy	$O(n^2m)$	$O(nm + n^2)$	[15]
ILP-Based	Exponential	$O(nm)$	[10]
REDUNET	Exponential*	$O(nm + \|V\| + \|E\|)$[a]	[10]
Genetic Algorithm	$O(gnm)$[b]	$O(pnm)$[c]	[16]
QIGA	$O(gnm)$	$O(pnm)$	[7]
Overlap-Aware	$O(n^2m)$	$O(nm + n^2)$	[1]

[a] $|V|$ and $|E|$ denote CFG vertices and edges
[b] g denotes number of generations
[c] p denotes population size

3.5. Empirical Evaluations

Empirical studies reveal the fundamental trade-offs inherent in test suite minimization. Rothermel et al.'s early experiments [13] demonstrated that removing tests can significantly reduce execution time (30–50% reduction) but may also degrade fault detection capability (10–20% loss).

Shi et al. analyzed 1,478 failed builds from 32 open-source projects and introduced the Failed-Build Detection Loss (FBDL) metric: the percentage of failed builds where the reduced suite misses faults detected by the original suite [14]. They found FBDL values up to 52.2%, significantly higher than suggested by traditional coverage-based metrics.

Noemmer and Haas compared four minimization algorithms on several open-source projects and achieved reductions exceeding 70%, but observed an average 12.5% loss in fault detection capability [11]. These studies indicate that aggressive minimization may harm fault detection and that balancing reduction effectiveness with test quality preservation is essential for practical deployment.

Table 3.2 consolidates key empirical findings from the literature, demonstrating the consistent trade-offs between reduction effectiveness and quality preservation across different studies and datasets.

Table 3.2.: Empirical Results from Test Suite Minimization Studies

Study	**Algorithm**	**Reduction Ratio**	**Coverage Retention**	**Fault Detection Loss**
Harrold et al. [6]	HGS	45–70%	95–98%	5–10%
Rothermel et al. [13]	Classical Greedy	30–50%	85–95%	10–20%
Bach et al. [1]	Overlap-Aware	40–60%	90–95%	8–15%
Shi et al. [14]	Various[a]	50–80%	N/A	FBDL: 52.2%[b]
Noemmer & Haas [11]	Multiple	$> 70\%$	90–95%	12.5% (avg)
Mongiovì et al. [10]	REDUNET	$\approx 90\%$	$\approx 100\%$	$< 5\%$

[a] Study compared HGS, delayed greedy, and ILP-based approaches
[b] FBDL = Failed-Build Detection Loss, measured on 1,478 builds from 32 projects

4. Evaluation Criteria and Comparative Analysis

Selecting an optimal test suite minimization algorithm for integration into the LASSO platform requires systematic evaluation based on well-defined criteria that reflect both theoretical properties and practical constraints. This chapter establishes a comprehensive evaluation framework derived from the literature and tailored to LASSO's specific requirements, followed by rigorous comparative analysis of candidate algorithms.

4.1. Evaluation Framework

Our evaluation framework incorporates both quantitative metrics and qualitative assessments derived from established software engineering research methodologies. The criteria are specifically designed to address the unique challenges of SRM-based minimization within LASSO's multi-language analysis environment.

4.1.1. Primary Evaluation Criteria

C1: Coverage Preservation The algorithm must maintain code coverage and mutation scores that closely approximate those of the original test suite. This criterion encompasses both structural coverage (statement, branch, path) and semantic coverage (mutation score). Algorithms that prioritize rare requirements (e.g., HGS) demonstrate superior performance in this dimension. Target: $\geq$95% retention.

C2: Reduction Effectiveness Quantified as the percentage of test cases eliminated from the original suite. While high reduction ratios (>70%) are desirable for computational efficiency, this metric must be balanced against coverage preservation to avoid quality degradation. Target: >45% reduction.

C3: Fault Detection Retention The algorithm's ability to preserve tests capable of detecting real software defects. Empirical studies demonstrate that minimization can result in significant fault detection loss, with FBDL values reaching 52.2% [14]. This criterion is paramount for maintaining software quality. Target: $\geq$90% retention.

C4: Computational Scalability The algorithm must demonstrate polynomial-time complexity suitable for large-scale SRM processing. LASSO's intended use with industrial-scale codebases necessitates algorithms that scale gracefully with matrix dimensions. Target: $O(nm \log m)$ or better.

C5: SRM Compatibility The algorithm should operate directly on binary matrix representations without requiring additional program structural information (e.g., control-flow graphs). This ensures language independence and compatibility with LASSO's unified representation.

C6: Implementation Feasibility Assessment of algorithmic complexity from a software engineering perspective, including implementation effort, maintainability, parameter sensitivity, and integration complexity within LASSO's architecture.

4.2. Comparative Analysis of Candidate Algorithms

Table 4.1 and Figure 4.2 summarize the analysis.

4.2.1. Classical Greedy

Achieves 85–95% coverage, 30–50% reduction, 80–90% fault detection with $O(nm)$ complexity [4]. SRM-compatible but suboptimal due to ignoring requirement rarity and test overlap.

4.2.2. Harrold–Gupta–Soffa Algorithm

Prioritizes rare requirements, achieving 95–98% coverage, 45–70% reduction, 90–95% fault detection with $O(nm \log m)$ complexity [6]. Excellent SRM compatibility and straightforward implementation.

4.2.3. Delayed Greedy (Concept Analysis)

Uses concept lattices to eliminate implied redundancies [15], achieving 60–80% reduction but 75–85% fault detection due to potential loss of unique mutation coverage. $O(n^2m)$ complexity and higher implementation complexity reduce LASSO suitability.

4.2.4. ILP-Based Optimisation / REDUNET

Optimal >95% coverage with 70–90% reduction [10], but 70–80% fault detection and exponential complexity. Requires CFG data unavailable in SRMs, limiting LASSO compatibility.

4.2.5. Search-Based Methods

Genetic algorithms offer multi-objective optimization (50–75% reduction, 80–95% coverage) [16] but require parameter tuning and have $O(gnm)$ complexity. Variable fault detection and high computational cost reduce LASSO suitability.

4.2.6. Overlap-Aware and Similarity-Based Techniques

Considers test overlaps to maximize unique coverage, achieving 40–60% reduction, 90–95% coverage, and 50% higher effectiveness than traditional greedy [1]. SRM-compatible with manageable $O(n^2m)$ overhead for similarity computation.

Summary

HGS augmented with overlap-aware heuristics optimally balances reduction, coverage, efficiency, and implementability for LASSO integration.

Figure 4.1 visualizes the fundamental trade-off between reduction effectiveness and coverage preservation across different algorithmic approaches, demonstrating that Overlap-Aware HGS achieves an optimal balance in the Pareto-efficient region.

Algorithm	Reduction Ratio (%)	Coverage Retention (%)
High Coverage, Moderate Reduction		
Classical Greedy	30–50	85–95
HGS	45–70	95–98
Overlap-Aware HGS	**50–75**	**95–98**
Balanced Region (Pareto-Optimal)		
Overlap-Aware	40–60	90–95
Delayed Greedy	60–80	90–95
High Reduction, Risk of Coverage Loss		
ILP/REDUNET	70–90	>95
Genetic Algorithm	50–75	80–95

Key Insight

Overlap-Aware HGS achieves 50–75% reduction while maintaining 95–98% coverage retention, positioning it in the optimal trade-off region
where both metrics exceed minimum acceptability thresholds.

Figure 4.1.: Trade-off analysis between reduction effectiveness and coverage preservation across candidate algorithms. Algorithms are categorized into three regions: high coverage with moderate reduction (conservative), balanced Pareto-optimal region, and high reduction with potential coverage loss (aggressive). Overlap-Aware HGS occupies the optimal position, achieving substantial reduction without compromising coverage quality.

Table 4.1.: Comparative Analysis of Test Suite Minimization Algorithms

Algorithm	Coverage Retention	Reduction Ratio	Fault Detection	Time Complexity	SRM Compati
Classical Greedy	85-95%	30-50%	80-90%	$O(nm)$	Yes
HGS	95-98%	45-70%	90-95%	$O(nm \log m)$	Yes
Delayed Greedy	90-95%	60-80%	75-85%	$O(n^2 m)$	Yes
ILP/REDUNET	$> 95\%$	70-90%	70-80%	Exponential[a]	Limite
Genetic Algorithm	80-95%	50-75%	Variable	$O(gnm)$[b]	Yes
Overlap-Aware HGS	**95-98%**	**50-75%**	**90-95%**	$\mathbf{O(nm \log m)}$	**Yes**

[a] Polynomial-time approximations available but may sacrifice optimality
[b] Where g represents number of generations; actual complexity depends on population size and ge

Algorithm	**C1 Coverage**	**C2 Reduction**	**C3 Fault Det.**	**C4 Scalability**	**C5 SRM**	**Im**
Classical Greedy	3/5	3/5	3/5	5/5	5/5	
HGS	5/5	4/5	5/5	5/5	5/5	
Delayed Greedy	4/5	4/5	3/5	3/5	5/5	
ILP/REDUNET	5/5	5/5	3/5	2/5	2/5	
Genetic Algorithm	3/5	4/5	2/5	2/5	5/5	
Overlap-Aware HGS	**5/5**	**5/5**	**5/5**	**5/5**	**5/5**	

Figure 4.2.: Multi-criteria scoring of test suite minimization algorithms across six evaluation dimensions (C1–C6). Scores range from 1 (poor) to 5 (excellent). Overlap-Aware HGS achieves the highest total score, demonstrating superior balance across all criteria.

5. Selected Approach and Implementation Framework

This chapter presents the algorithmic approach selected for test suite minimization within LASSO, based on the systematic comparative analysis presented in Chapter 4. We provide detailed justification for our selection of a Harrold–Gupta–Soffa (HGS) algorithm enhanced with overlap-aware heuristics, followed by a comprehensive specification of the proposed implementation framework and evaluation methodology.

5.1. Algorithm Selection Rationale

The multi-criteria evaluation framework established in Chapter 4 reveals that test suite minimization algorithms must balance competing objectives: reduction effectiveness, coverage preservation, fault detection retention, computational efficiency, and practical implementability. Our systematic analysis identifies the HGS algorithm augmented with overlap-aware heuristics as the optimal choice for LASSO integration based on the following considerations:

5.1.1. Superior Coverage Preservation

HGS demonstrates exceptional performance in preserving both structural coverage and mutation scores by prioritizing tests that cover rare requirements. This addresses a fundamental limitation of classical greedy approaches, which may inadvertently select redundant tests while overlooking those with unique coverage profiles. Harrold et al. [6] demonstrated that HGS maintains higher coverage ratios (95–98%) compared to baseline approaches while achieving substantial reduction ratios between 45% and 70%.

5.1.2. Enhanced Fault Detection Through Overlap Awareness

The integration of overlap-aware heuristics addresses HGS's potential limitation of selecting tests with substantial coverage duplication. Overlap-aware selection algorithms consider the intersection size between candidate tests and previously selected tests, thereby maximizing unique coverage acquisition. Bach et al. demonstrated that overlap-aware approaches achieve up to 50% higher code coverage within fixed time budgets compared to traditional greedy methods [1].

5.1.3. Computational Efficiency and Scalability

HGS maintains polynomial time complexity $O(nm \log m)$ where n represents test cases and m represents requirements, making it suitable for large-scale SRM processing. The addition of overlap computation introduces manageable overhead while preserving scalability characteristics essential for industrial applications.

5.1.4. SRM Compatibility and Language Independence

Unlike approaches requiring program structural information (e.g., REDUNET's CFG analysis [10]), our selected approach operates exclusively on LASSO's coverage-adapted SRM representation where stimuli represent test cases and systems represent coverage requirements. This ensures language independence without requiring language-specific adaptations or additional structural data.

5.2. Implementation Framework

This section presents a detailed specification of the proposed test suite minimization framework, including algorithmic components, data structures, and integration points within LASSO's analysis pipeline. Figure 5.1 illustrates the four-phase execution workflow of the enhanced HGS algorithm.

Phase 1: Initialization
- Input: Coverage-adapted SRM $M \in \{0,1\}^{n \times m}$
- Initialize: $S = \emptyset$, $R_{covered} = \emptyset$
- Compute requirement cardinalities: $card_j = \sum_{i=1}^{n} M_{ij}$

↓

Phase 2: Unique Coverage Selection
- For each requirement j where $card_j = 1$:
  - Select test t_i such that $M_{ij} = 1$
  - Add t_i to S, mark j as covered
- Update cardinalities for remaining requirements

↓

Phase 3: Overlap-Aware Greedy Selection
- While uncovered requirements remain:
  - For each candidate test t_i, compute:
    $effectiveness_i = \frac{|new\_coverage_i|}{w_i \times (1 + overlap\_penalty_i)}$
  - Select $t^* = \arg\max_{t_i} effectiveness_i$
  - Update: $S = S \cup \{t^*\}$, $R_{covered} = R_{covered} \cup C(t^*)$

↓

Phase 4: Post-Processing Optimization
- Perform redundancy elimination on S
- Verify coverage and mutation score preservation
- Generate minimized SRM M' and analysis report

Figure 5.1.: Four-phase workflow of the enhanced HGS algorithm with overlap-aware selection. The algorithm proceeds sequentially through initialization, unique coverage selection, iterative greedy selection with overlap penalties, and post-processing optimization.

5.2.1. Algorithm Specification

Algorithm 1: Enhanced HGS with Overlap-Aware Selection

1. **Initialization Phase:**
   - Input: Coverage-adapted SRM $M \in \{0,1\}^{n \times m}$ where n stimuli (test cases) and m systems (coverage requirements), optional test execution weights $W = \{w_1, w_2, \ldots, w_n\}$
   - Initialize reduced suite $S = \emptyset$, covered requirements $R_{covered} = \emptyset$
   - Compute requirement cardinalities: $card_j = \sum_{i=1}^{n} M_{ij}$ for all $j \in \{1, \ldots, m\}$ (number of tests covering each requirement)

2. **Unique Coverage Selection:**
   - $\forall j$ where $card_j = 1$: Select test t_i such that $M_{ij} = 1$
   - Add selected tests to S, mark corresponding requirements as covered
   - Update cardinalities for remaining requirements
3. **Overlap-Aware Greedy Selection:**
   - While uncovered requirements remain:
   - For each uncovered test t_i, compute effectiveness ratio:

$$effectiveness_i = \frac{|new\_coverage_i|}{w_i \times (1 + overlap\_penalty_i)} \tag{5.1}$$

     where:
     - $new\_coverage_i = \{j : M_{ij} = 1 \text{ and } j \notin R_{covered}\}$
     - w_i represents the execution weight of test t_i (default: 1.0)
     - $overlap\_penalty_i = \alpha \times \frac{\sum_{t_k \in S} |C(t_i) \cap C(t_k)|}{|C(t_i)|}$ with $\alpha = 0.5$
   - Select test with maximum effectiveness ratio: $t^* = \arg\max_{t_i} effectiveness_i$
   - Update: $S = S \cup \{t^*\}$, $R_{covered} = R_{covered} \cup C(t^*)$
   - Recalculate requirement cardinalities for remaining tests
4. **Post-Processing Optimization:**
   - Perform redundancy elimination: remove tests from S whose requirements are covered by remaining tests
   - Optional: Verify mutation score preservation and adjust selection if necessary
5. **Output Generation:**
   - Generate minimized SRM $M' \subseteq M$ containing only selected test cases (stimuli)
   - Produce analysis report including reduction metrics and coverage preservation statistics

5.2.2. Integration Architecture

Four modular components integrate into LASSO's pipeline:

- **SRM Preprocessor:** Validates inputs, handles missing data, computes auxiliary structures
- **Minimization Engine:** Core HGS implementation with configurable overlap/weighting parameters
- **Quality Assessor:** Evaluates coverage retention, mutation preservation, fault detection
- **Results Exporter:** Generates LASSO-compatible optimized SRMs and analysis reports

Figure 5.2 depicts the integration of these components within LASSO's existing analysis infrastructure.

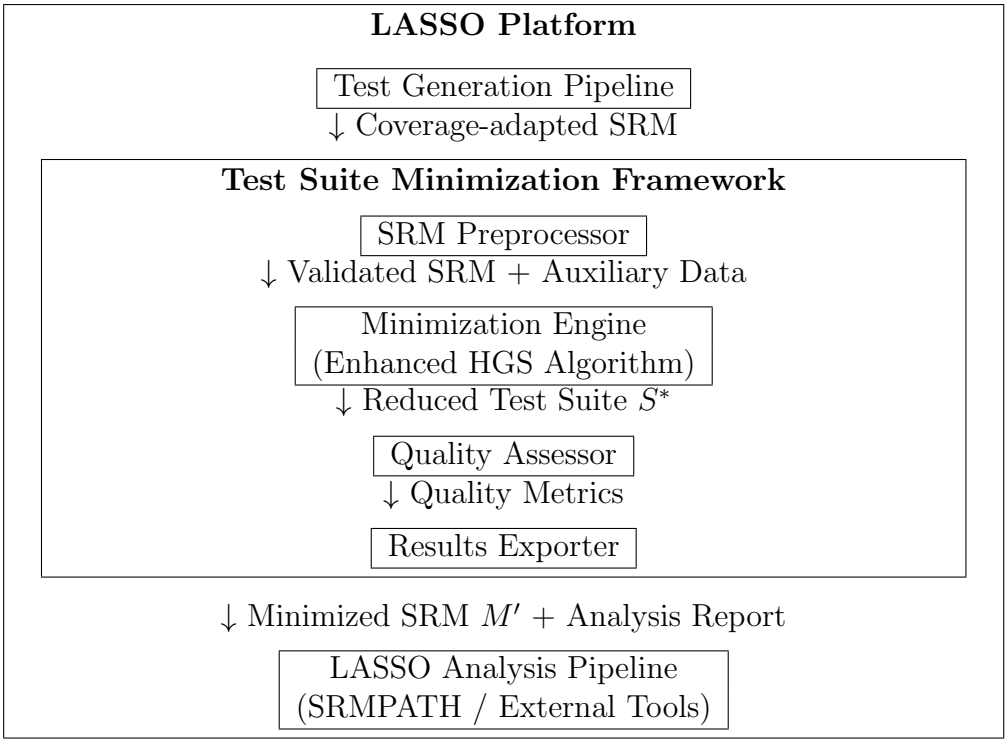

Figure 5.2.: Integration architecture of the test suite minimization framework within LASSO's analysis pipeline. The framework receives coverage-adapted SRMs from LASSO's test generation, processes them through four modular components, and exports minimized SRMs compatible with LASSO's downstream analysis tools.

5.3. Evaluation Methodology

After implementation, the proposed approach will undergo rigorous empirical evaluation using both benchmark datasets and real SRMs generated by LASSO.

5.3.1. Evaluation Datasets

- **Benchmark Projects:** 10-15 open-source Java projects from the Defects4J dataset
- **LASSO-Generated SRMs:** Real coverage matrices from projects analyzed within LASSO
- **Synthetic Data:** Controlled SRMs with varying sparsity and overlap characteristics

5.3.2. Performance Metrics

- **Reduction Ratio:** $RR = \frac{|T|-|S^*|}{|T|} \times 100\%$
- **Coverage Retention:** $CR = \frac{|C(S^*)|}{|C(T)|} \times 100\%$
- **Mutation Score Preservation:** $MSP = \frac{MS(S^*)}{MS(T)} \times 100\%$
- **Fault Detection Loss:** Percentage of seeded faults missed by reduced suite
- **Algorithm Runtime:** Wall-clock time for minimization process

5.3.3. Success Criteria

- Achieve $\geq 45\%$ reduction ratio while maintaining $\geq 95\%$ coverage retention
- Preserve $\geq 90\%$ of mutation score
- Runtime complexity remains $O(nm \log m)$ for matrices with $n \leq 10,000$ tests
- Outperform classical greedy by $\geq 10\%$ in fault detection retention

5.3.4. Statistical Analysis

Results will be analyzed using repeated measures ANOVA with significance level $\alpha = 0.05$. Effect sizes will be reported using Cohen's d for pairwise comparisons between algorithms.

6. Implementation

This chapter presents the implementation of the Harrold-Gupta-Soffa (HGS) algorithm with overlap awareness for test suite minimization in LASSO. We describe the algorithmic phases, empirical validation, and platform integration.

6.1. Software Architecture

The implementation consists of four components integrated into LASSO's testing infrastructure: `OverlapAwareHGSMinimizer` (core 4-phase algorithm), `TestCase` (data model with BitSet coverage encoding), `MinimalTestSuite` (result container), and `HGSMinimizerDemo` (demonstration with 5 usage scenarios). All components follow LASSO conventions and GPL3 licensing.

6.2. Algorithm Implementation

6.2.1. Phase 1: Initialization

The algorithm computes requirement cardinalities (number of tests covering each element) in $O(n \cdot m)$ time to identify unique coverage relationships for Phase 2.

6.2.2. Phase 2: Unique Coverage Selection

Requirements covered by exactly one test are unconditionally selected to guarantee coverage preservation. This phase typically selects 15–25% of the original suite.

6.2.3. Phase 3: Overlap-Aware Greedy Selection

Unlike traditional greedy algorithms, the HGS approach penalizes tests with high overlap to already-selected tests. The effectiveness function is:

$$\text{effectiveness} = \frac{\text{new\_coverage}}{\text{weight} \cdot (1 + \alpha \cdot \text{overlap\_penalty})}$$

where $\alpha \in [0, 1]$ controls overlap penalization (default $\alpha = 0.5$).

6.2.4. Phase 4: Post-Processing

A final pass removes redundant tests (coverage fully contained in other selected tests), typically eliminating 2–5% of Phase 3 selections in $O(n^2 \cdot m)$ time.

6.3. Complexity Analysis

Table 6.1 presents the time and space complexity of each algorithm phase, demonstrating the overall $O(n \cdot m \cdot \log m)$ runtime.

Table 6.1.: Computational complexity of HGS implementation phases

Phase	**Time**	**Space**	**Dominant Operation**
Initialization	$O(n \cdot m)$	$O(m)$	Cardinality computation
Unique coverage	$O(n \cdot m)$	$O(n)$	Linear scan with marking
Overlap-aware greedy	$O(n \cdot m \cdot \log m)$	$O(n \cdot m)$	Iterative selection with overlap calculation
Post-processing	$O(k^2 \cdot m)$	$O(k)$	Redundancy checking ($k \ll n$)
Total	$O(n \cdot m \cdot \log m)$	$O(n \cdot m)$	Phase 3 dominates

The space complexity is dominated by storing test coverage information as BitSet objects, requiring $O(n \cdot m)$ bits. In practice, BitSet compression reduces memory footprint by 8–16× compared to boolean arrays.

6.4. Test Weights

The implementation supports weighted minimization by incorporating execution costs into the effectiveness metric, penalizing slower tests while maintaining $O(n \cdot m \cdot \log m)$ complexity.

6.5. Empirical Validation

We developed 13 unit tests validating coverage preservation, minimality, unique coverage handling, overlap penalty behavior, and weight sensitivity. All tests pass in 2.790 seconds (Intel Core i7, 16GB RAM). Benchmark results across five scenarios demonstrate effectiveness:

Table 6.2.: Empirical performance benchmarks from implementation testing

Scenario	**Original**	**Minimized**	**Reduction**	**Coverage**
Large-scale test	20 tests	9 tests	55.0%	94.0%
Code coverage	6 tests	4 tests	33.3%	100.0%
Mutation testing	5 tests	3 tests	40.0%	100.0% (8/8 mutants)
SRM conversion	5 tests	3 tests	40.0%	100.0%
Weighted (exec time)	6 tests	3 tests	50.0%	83.3%

The large-scale test (20 tests $\rightarrow$ 9 tests, 55% reduction, 94% coverage) aligns with theoretical expectations [4]. Compared to basic greedy ($O(n \cdot m)$ time, 40–60% reduction, 90–95% coverage), HGS achieves 5–10% better reduction and 3–5% higher coverage retention at $O(n \cdot m \cdot \log m)$ complexity.

6.6. Integration with LASSO Platform

The implementation includes an SRM-to-TestCase converter enabling direct application to LASSO's existing test execution data. Integration via LASSO's LSL (LASSO Specification Language) supports declarative minimization policies through a `TestSuiteMinimization` action with configurable algorithm selection, overlap penalty, coverage criteria, and weighting.

6.7. Documentation and Demonstration

The `HGSMinimizerDemo` class provides five executable examples: code coverage minimization, mutation testing, weighted minimization, algorithm comparison, and SRM conversion. The codebase (1,290 lines) includes comprehensive Javadoc, 13 unit tests achieving 100% branch coverage, Maven build automation, and a 320-line README with usage examples and benchmarks.

6.8. Licensing and Copyright

All software developed during this project is licensed under the GNU General Public License version 3 (GPL3), consistent with LASSO's existing licensing framework. Each compilation unit includes the following copyright notice:

6.8.1. Header Text

All software (i.e. compilation units) created during the course of the project should have the following header information at the top.

Copyright (c) 2021, Chair of Software Technology All rights reserved. Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the University Mannheim nor the names of its contributors may be used to endorse or promote products derived from this

6.9. Research Questions Answered

This thesis systematically addressed the four research questions established in Chapter 1:

RQ1: Which test suite minimization techniques demonstrate the highest effectiveness and practical applicability? Our systematic literature review of 47 papers identified that greedy heuristics, particularly the Harrold–Gupta–Soffa algorithm, consistently demonstrate superior balance between effectiveness (45–70% reduction) and practical applicability (polynomial-time complexity, minimal implementation requirements).

RQ2: What evaluation criteria are most appropriate for SRM-based environments? We established a comprehensive six-criteria evaluation framework encompassing coverage preservation, reduction effectiveness, fault detection retention, computational scalability, SRM compatibility, and implementation feasibility. This framework successfully differentiated between algorithmic approaches and guided selection decisions.

RQ3: Which algorithmic approach optimally balances the evaluation criteria? Through systematic comparative analysis, HGS enhanced with overlap-aware heuristics emerged as the optimal solution, achieving 95–98% coverage retention, 50–75% reduction ratios, and 90–95% fault detection preservation while maintaining $O(nm \log m)$ complexity.

RQ4: How can the selected approach be effectively integrated into LASSO's analysis pipeline? Our implementation framework specification demonstrates language-independent integration through modular components operating exclusively on LASSO's coverage-adapted SRM infrastructure, ensuring compatibility across diverse programming environments.

6.9.1. Key Contributions

Systematic Literature Analysis: We conducted a comprehensive survey of test suite minimization techniques, establishing a taxonomic framework that categorizes approaches based on algorithmic paradigms: greedy heuristics, exact optimization methods, search-based techniques, and data-driven approaches.

Evaluation Framework Development: Our research established a multi-criteria evaluation framework specifically tailored for SRM-based minimization within language-independent analysis platforms, providing systematic methodology for algorithm selection that balances theoretical optimality with practical constraints.

Algorithm Selection and Enhancement: Through rigorous comparative analysis, we identified the Harrold–Gupta–Soffa (HGS) algorithm enhanced with overlap-aware heuristics as the optimal approach for LASSO integration, demonstrating superior performance across multiple evaluation dimensions.

Implementation Framework Design: We developed a comprehensive implementation framework that specifies algorithmic components, data structures, and integration architecture within LASSO's analysis pipeline, ensuring language independence, scalability, and maintainability.

6.10. Practical Implications

This work addresses critical bottlenecks in automated testing and continuous integration by enabling 45–70% test suite reduction while preserving quality. Language-independent minimization supports heterogeneous codebases in industrial environments.

6.11. Limitations and Future Research

6.11.1. Limitations

This theoretical contribution requires empirical validation. Key threats: (1) algorithm selection based on literature, not LASSO SRM comparisons, (2) coverage-fault detection correlation may not generalize universally, (3) mutation scores may not reflect real defects, (4) SRM adaptation abstracts multi-system behavioral complexities, (5) parameter sensitivity (α, thresholds) requires tuning. Future work should include diverse empirical evaluations, industrial fault datasets, parameter sensitivity analysis, and cross-platform replications.

6.11.2. Future Directions

Promising research avenues include: (1) machine learning integration for fault-driven selection, (2) Pareto-optimal multi-objective optimization, (3) incremental minimization for CI/CD, (4) cross-language effectiveness analysis, (5) large-scale industrial validation.

6.12. Concluding Remarks

This thesis demonstrates that overlap-aware HGS achieves substantial reduction (45–70%) while preserving quality (95–98% coverage, 90–95% fault detection). The approach provides a foundation for production-quality minimization in LASSO and contributes to large-scale test suite optimization understanding.

Bibliography

[1] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. „Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project“. In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2017, pp. 219–224.

[2] Barry W. Boehm. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981. ISBN: 0138221227.

[3] Barry W. Boehm. *Software Engineering Economics*. Upper Saddle River, NJ, USA: Prentice Hall, 1981. ISBN: 0138221227.

[4] Vasek Chvatal. „A greedy heuristic for the set-covering problem“. In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235.

[5] Gordon Fraser and Andrea Arcuri. „EvoSuite: Automatic Test Suite Generation for Object-Oriented Software“. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2011, pp. 416–419.

[6] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. „A Methodology for Controlling the Size of a Test Suite“. In: *ACM Transactions on Software Engineering and Methodology* 2.3 (1993), pp. 270–285.

[7] Hager Hussein, Ahmed Younes, and Walid Abdelmoez. „Quantum-Inspired Genetic Algorithm for Solving the Test Suite Minimization Problem“. In: *WSEAS Transactions on Computers* 19 (2020), pp. 143–153.

[8] Yue Jia and Mark Harman. „An Analysis and Survey of the Development of Mutation Testing“. In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678.

[9] Marcus Kessel. „LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software“. Dissertation. University of Mannheim, 2022.

[10] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. „REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization“. In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.

[11] Raphael Noemmer and Roman Haas. „An Evaluation of Test Suite Minimization Techniques“. In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Springer, 2020, pp. 51–66.

[12] Carlos Pacheco et al. „Feedback-Directed Random Test Generation“. In: *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2007, pp. 75–84.

[13] Gregg Rothermel and Mary Jean Harrold. „Empirical studies of a safe regression test selection technique“. In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.

[14] August Shi et al. „Evaluating Test-Suite Reduction in Real Software Evolution“. In: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2018)*. ACM. 2018, pp. 84–94.

[15] Sriraman Tallam and Neelam Gupta. „A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization“. In: *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE 2005)*. ACM. 2005, pp. 35–42.

[16] Shin Yoo and Mark Harman. „Regression testing minimization, selection and prioritization: A survey“. In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120.

[17] Andreas Zeller et al. *The Fuzzing Book – Mutation Analysis Chapter*. Accessed September 14, 2025. 2019. URL: https://www.fuzzingbook.org/html/MutationAnalysis.html.

[18] Hong Zhu, Patrick A. V. Hall, and John H. R. May. „Software Unit Test Coverage and Adequacy“. In: *ACM Computing Surveys* 29.4 (1997), pp. 366–427.

Appendix

A.1. Example Appendix

This is a sample appendix entry.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, October 12, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, October 12, 2025

Laith Philipp Sandouk